Technical Design Document: Longevity Dashboard Modernisation

Date: March 14, 2026

Proposed Architecture

Executive Summary

This document describes the Technical Architecture of a Longevity Dashboard System. Development of this new system will be from the ground up, ensuring that it can scale within an Enterprise Environment. The existing Monolithic Flask Application and Excel Data Store will no longer be used. The existing code will be used only as a reference blueprint for developing new business logic. The new Longevity Dashboard System will also solve critical issues of coupling, scalability, and synchronous performance bottlenecks.

1. Current System Analysis

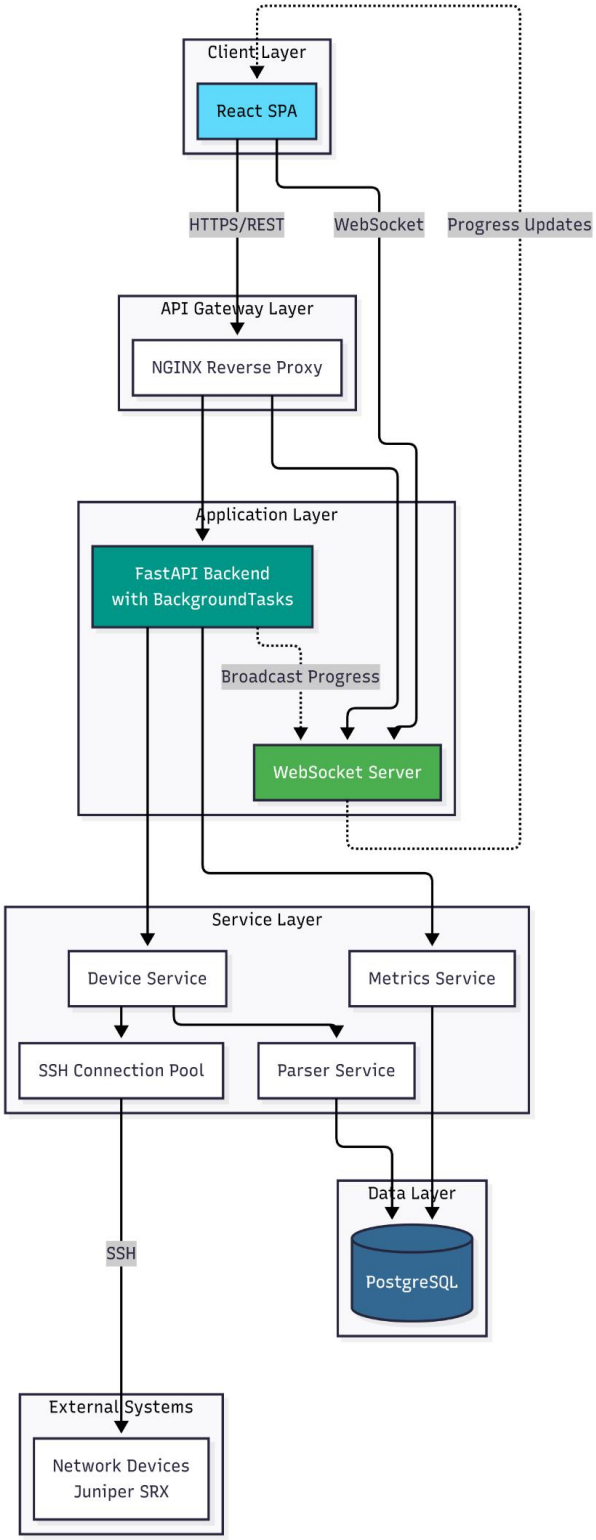
1.1 Current/Existing System Architecture (Blueprint for reference only)

The existing architecture is:

- **Backend:** Flexible and scalable Flask web application (app.py); supports Application Programming Interface (API) calls and HTML template generation.
- **Business Logic:** Single monolithic application (Longevity.py) provides management for Secure Shell client connections, issues commands, parses results, and persistently stores all data associated.
- **Frontend:** Server-side HTML (using standard HTML) generated by Flask application with standard JavaScript on clients to perform validation.
- **Data Storage:** Data is currently stored using an Excel spreadsheet (security_monitoring.xlsx) and uses a file lock.
- **Configuration:** Configuration is currently contained in a JSON file (data.json) with hard-coded username and password.

2. Proposed Architecture

2.1 System Architecture Diagram



2.2 Technology Stack Comparison

Component	Previous	Proposed	Justification for Solo Execution
Backend Framework	Flask	FastAPI	Async support, automatic OpenAPI docs, and type safety drastically reduce debugging time for a solo dev.
Frontend	Vanilla JS	React	Component reusability and a vast ecosystem allow rapid UI development.
Database	Excel files	PostgreSQL	ACID compliance, concurrent access, scalability.
Real-time Updates	None	WebSocket	Live progress updates, reduced server load vs legacy polling.
Task Queue	None	BackgroundTasks	Built-in async execution removes the need to manage external message brokers.
Containerization	None	Docker Compose	Local orchestration guarantees it works on the server exactly as it does locally.

3. Detailed Component Design

3.1 Backend API (FastAPI)

Why FastAPI over Flask:

- Supports async/await natively so that you can do multiple SSH requests at once.
- Has Pydantic, which allows for automatic request and response validation.
- Built-in OpenAPI documentation generation.
- Type hints allow for better IDE assistance and a lower chance of runtime errors.

3.2 Frontend (React)

Reasons to use React instead of Server Side Rendering:

- Component-based architecture for better code reuse.
- A virtual DOM enables fast user interface updates for real-time data.
- Native support to use WebSocket via custom hooks for live updates to the user interface.

Key Features:

- Real-time metrics update using a WebSocket connection.
- Progress notifications for the collection of metrics in real-time (example: "Device 3 of 10: Connecting") with the ability to see live progress.
- Support for responsive table formats for displaying metrics and other data types.
- Ability to export metric and other data types (CSV, JSON, PDF).

3.3 Database Schema (PostgreSQL)

Reasons to use PostgreSQL instead of Excel:

- **Concurrency:** MVCC provides for multiple readers and writers to work on the same data simultaneously without any locking.
- **Reliability:** ACID transactions and write-ahead logging (WAL) will recover data from a crash.
- **Scalability:** In addition to performing partitioning, replication, and sharding, PostgreSQL can handle large volumes of data and provide for efficient and effective methods for data storage and management.

3.4 Service Layer Design

We will develop a new service layer using modular services instead of migrating the old service of Longevity.py. The old code will only be used for reference to develop the required business

logic and regex patterns.

3.4.1 Device Service

Responsibilities: Device registration, lifecycle management, and device type classification.

Reference Implementation: Device configuration logic previously hardcoded in data.json.

3.4.2 Fast SSH Connection Service (Sub-1-Minute Optimisation)

The fast SSH connection service has two responsibilities: managing fast connections (as fast as possible) to reduce overall collection time. This service has a sub-1-minute collection time optimisation strategy that does two things:

Sub-1-Minute Strategy:

1. **Use true concurrency (asyncio):** Use non-blocking I/O (asyncssh or other equivalent) to poll all 10 devices simultaneously (poll all devices at the same time and do not wait for one device to be polled before moving onto the next) rather than including the total poll time for each device in the overall poll time (the total poll time will be equal to the slowest device to be polled).
2. **Use persistent connection pooling:** Maintain keep-alive authenticated tunnels so that there is no expensive TCP/SSH handshake on every refresh.

The reference implementation for this service is the blocking get_handle() and login_and_run_commands() functions.

3.4.3 Command Execution Service

The command execution service has two responsibilities: Execute Device Type Specific Commands and Buffer Output From Executed Commands.

The reference implementation is run_commands_vsrx() and run_commands_highend().

3.4.4 Parser Service

Responsibilities: Output Parsing Via Regex, Trusting Your Data, And Finding Errors.

Reference Implementation: parse_security_monitoring(), parse_system_core_dumps(), (To extract legacy regex strings. The parsing engine will be completely rewritten).

3.4.5 Metrics Service

Responsibilities: Metrics Aggregation And Historical Data Queries.

Reference Implementation: saving legacy data to Excel will now use Robust SQLAlchemy

ORM.

3.5 Web Socket Updates & Async Task Processing MVP

- FastAPI BackgroundTask (Built-in) Asynchronous Process.
- Web Sockets for Real-Time Communication to Update Progress
- No Additional Msg Broker Element is Needed for V1.

WebSocket Implementation (FastAPI):

```
@app.websocket("/ws/{job_id}")
async def websocket_endpoint(websocket: WebSocket, job_id: str):
    await websocket.accept()
    # Send updates on the status of the job to all connected users
    async for progress in job_progress_stream(job_id):
        await websocket.send_json(progress)
```

Task Flow:

1. User clicks on "Refresh Metrics," gets a job_id returned from the API.
2. Frontend App connects via Web Sockets to: /ws/{job_id}.
3. BackgroundTask asynchronously begins running SSH commands in parallel on all devices.
4. Send status updates of progress to all users connected via Web Sockets (Ex: Device (1/10) collecting metrics...)
5. Store results in PostgreSQL.
6. Display updated metrics to the UI in real-time.

4. Scalability & Deployment

4.1 Horizontal Scaling Strategy

Component	Scaling Method	Trigger
FastAPI	Kubernetes HPA	CPU > 70% or Request Rate > 1000/s
PostgreSQL	Read replicas	Read latency > 50ms

4.2 Containerization (Docker)

Benefits: Dev, Staging & Production environments are always the same; simplify dependency management; resource isolation; roll-back capabilities

5. Security Enhancements

Current Risk	Mitigation Strategy
Hardcoded credentials in data.json	Environment variables / HashiCorp Vault
No authentication	JWT-based authentication with role-based access control
Plain HTTP	TLS/HTTPS everywhere, certificate management
No audit logging	Comprehensive audit trail in the database

6. Success Metrics

Metric	Current Legacy	MVP Target	Measurement
Concurrent device monitoring	~5	20+	Load test results
API response time (p95)	N/A	<500ms	System metrics
Data collection time (10 devices)	~5 minutes	**< 1 minute**	Job completion logs
System uptime	Unknown	99%	Uptime monitoring

7. Conclusion

The new design changes the Longevity Dashboard from being a single software product to becoming a progressive, flexible framework that is designed to fulfil enterprise high-volume demands. The process begins by taking into consideration a new architecture that incorporates best practices for clean and modifiable structures. By leveraging the prior data-based product simply as a reference, the proposed architecture addresses concerns with Excel locking and synchronous performance.

~ Shivang Singh